

	PROGRAMA OFICIAL DE CURSO (Pregrado y Posgrado)
	UNIVERSIDAD DE ANTIOQUIA

1. INFORMACIÓN GENERAL			
Nombre del Curso:	ANÁLISIS Y DISEÑO DE SISTEMAS II		
Programa académico al que pertenece:	ING DE SISTEMAS VIRTUAL		
Unidad Académica:	Facultad de Ingeniería		
Vigencia:	2024-1 2024-2	- Código curso:	2554506
Tipo de curso:	Profesional		
CARACTERÍSTICAS DEL CURSO			
Habitable (H):	NO	Validable (V):	NO
Clasificable (C):	NO	Evaluación de suficiencia (Posgrado):	NO
Modalidad educativa del curso:	Virtual		
Área, núcleo o componente de la organización curricular a la que pertenece el curso	Básicas		
Número de créditos académicos:	3		
Horas totales de interacción estudiante-profesor:	96	Horas totales de trabajo independiente:	48
Horas totales del curso del semestre:	144		
Horas totales de actividades académicas teóricas:	0	Horas totales de actividades académicas prácticas:	0
Horas totales de actividades académicas teórico-prácticas:	96		

PROGRAMAS ACADÉMICOS EN LOS CUALES SE OFRECE EL CURSO	
506 - INGENIERÍA DE SISTEMAS - REGIÓN Versión: 5	
Pre-requisitos:	2554403 - ANÁLISIS Y DISEÑO DE SISTEMAS I
Co-requisitos:	Ninguno
552 - INGENIERÍA DE SISTEMAS VIRTUAL-SEDE MEDELLIN Versión: 1	
Pre-requisitos:	2554403 - ANÁLISIS Y DISEÑO DE SISTEMAS I
Co-requisitos:	Ninguno
506 - INGENIERÍA DE SISTEMAS - REGIÓN Versión: 4	
Pre-requisitos:	2554403 - ANÁLISIS Y DISEÑO DE SISTEMAS I
Co-requisitos:	Ninguno

2. RELACIONES CON EL PERFIL

El curso pretende desarrollar en el estudiante las competencias necesarias para identificar, comprender y aplicar conceptos, principios, técnicas y métodos para realizar análisis y diseño de sistemas de software reusables y mantenibles, en un proceso de desarrollo de software de calidad, desempeñándose con ética en su ejercicio profesional y su conducta personal.

3. INTENCIONALIDADES FORMATIVAS

Explicitar los elementos orientadores del curso de acuerdo con el diseño curricular del programa académico: problemas de formación, propósitos de formación, objetivos, capacidades, competencias u otros. Se escoge una o varias de las anteriores posibilidades de acuerdo con las formas de organización curricular del programa académico, que se declaran en el Proyecto Educativo de Programa.

Saber:

(SA3) Comprender, a medida que emergen, teorías métodos, técnicas y tecnologías para el desarrollo, operación y mantenimiento de software.

(SA5) Comprender conceptos, principios, teorías y métodos de desarrollo de software.

(SA3- SA5) Comprender conceptos y principios del análisis y diseño de sistemas software.

(SA3- SA5) Conocer la utilidad del lenguaje de modelado UML en un proceso de desarrollo de software.

(SA4) Comprender los conceptos y teorías que garantizan la construcción de software seguro.

Ser:

(SER1) Respetar la normatividad en las soluciones propuestas para el desarrollo, operación y mantenimiento del software.

(SER2) Ser respetuoso de los derechos de propiedad intelectual, la seguridad y la privacidad de la información en el desarrollo de proyectos de ingeniería de software.

(SER3) Ser persistente, curioso, creativo y arriesgado en la formulación de proyectos de ingeniería de software.

(SER4) Ser ético en su conducta como ingeniero de software.

(SER5) Ser capaz de integrarse a un equipo de desarrollo para producir software.

(SER7) Tener excelentes habilidades de comunicación oral, escrita y escucha.

(SER 8) Ser crítico frente a los problemas de ingeniería de software.

Hacer:

(HA1) Desarrollar componentes de software.

(HA2) Trabajar eficazmente en proyectos de ingeniería de software.

(HA5) Desarrollar software en diversos dominios de aplicación usando métodos y estándares internacionales.

(HA6) Modelar aspectos dinámicos y estáticos de un sistema en el contexto del desarrollo de software. Específicamente utilizar el lenguaje de modelado UML en el proceso de diseño de software.

(HA7) Evaluar la calidad de los productos de software bajo estándares internacionales.

(HA8) Diseñar y programar aspectos de seguridad que afectan la privacidad de la información en los proyectos de ingeniería de software.

(HA8 - HA9) Implementar un proyecto de software, donde:

Se apliquen los conceptos, principios y patrones de diseño de software.

Se cumplan con las restricciones del contexto de ejecución,
Se programen aspectos de seguridad que afectan la privacidad de la información.
(HA11) Comunicar con eficiencia temas y aplicaciones relacionadas con la ingeniería de software.

4. APORTES DEL CURSO A LA FORMACIÓN INTEGRAL Y A LA FORMACIÓN EN INVESTIGACIÓN

El curso permite identificar, comprender y aplicar conceptos, principios, técnicas y métodos para realizar análisis y diseño de sistemas de software reusables y mantenibles, en un proceso de desarrollo de software de calidad, desempeñándose con ética en su ejercicio profesional y su conducta personal.

5. DESCRIPCIÓN DE LOS CONOCIMIENTOS Y/O SABERES

A continuación se enuncian los ejes temáticos y conocimientos trabajados en el curso:

Unidad 1 - Diagnóstico, Scrum y Proyecto: (2 semanas)

- ☒ Evaluación diagnóstica
- ☒ Repaso marco metodológico SCRUM (o seleccionar alguna metodología ágil para adoptar)
- ☒ Sprint, backlog, reuniones, roles, item backlog (historia de usuario), tablero, WIP, etc.20 Wilmer
- ☒ Iniciar la identificación y definición del proyecto de desarrollo para el curso.
- ☒ Laboratorio de la unidad

Unidad 2 - Modelo de Análisis: (2 semanas)

- ☒ Modelado conceptual. Beneficios en el marco del desarrollo de software. -
- ☒ Elaboración y refinamiento del Modelo de dominio.
- ☒ Modelado en UML
- ☒ Identificación de la interfaz externa de un sistema: diagramas de secuencias del sistema DSS.
- ☒ Laboratorio de Modelado de Análisis

Unidad 3 - Introducción al Diseño arquitectural: (2 semanas)

- ☒ Introducción al Diseño arquitectural.
- ☒ Diseño de arquitecturas de software: Monolítica, Cliente / servidor, Orientadas a servicios, Basadas en componentes, Concurrentes y de tiempo real, Líneas de productos de software, Orientadas a objetos, Microservicios, N-Capas.
- ☒ Diseño de arquitecturas N-Capas (N-layers)
- ☒ Modelado en UML. (despliegue, paquetes, componentes)
- ☒ Laboratorio de Modelado de Arquitectura

Unidad 4 - Principios básicos de diseño: (3 semanas)

- ☒ Conceptos básicos para el diseño orientado a objetos: abstracción, encapsulamiento, cohesión, acoplamiento, herencia, polimorfismo, interfaces.
- ☒ Principios de diseño orientado a objetos: SOLID: Single Responsibility Principle, Open/

Closed Principle, Liskov Substitution Principle, Interface Segregation Principle, Dependency Inversion Principle.

☒ Laboratorio con casos de estudio.

Unidad 5 - Patrones de diseño y antipatrones: (5 semanas)

☒ Patrones GRASP

☒ Patrones GoF

☒ Modelado en UML: aspectos dinámicos y estáticos.

☒ Implementación de patrones.

☒ Antipatrones de diseño de software

☒ Laboratorio

Unidad 6 - Calidad en el diseño de software (2 semanas)

☒ Concepto de deuda técnica

☒ Código limpio

☒ Pruebas unitarias de software

6. METODOLOGÍA (SUGERIDA)

Estrategias didácticas:

Aprendizaje Basado en Proyectos (ABP), Aprendizaje invertido, Aprendizaje entre pares

Metodología(s) utilizada(s):

El curso se orienta principalmente en el Aprendizaje Basado en Proyectos (ABP) complementado con otras aproximaciones metodológicas. El ABP es una metodología activa que se centra en la resolución de un proyecto en el que se puedan aplicar los objetivos definidos, a través del abordaje de situaciones complejas en lugar de la simple transmisión de información..

Algunas características de la metodología del Aprendizaje Basado en Proyectos que se alinean con las actividades mencionadas son:

Desarrollo de un proyecto de software.

Desarrollo de actividades de aprendizaje basadas en la interacción grupal.

Elaboración de mapas mentales y conceptuales.

Trabajo en equipo.

talleres dictados por monitores y otros estudiantes.

Aplicación del aula inversa.

Análisis de casos de estudio.

Socializaciones y presentación de avances.

Retos de modelado y programación.

Utilización de herramientas de modelado.

Medios y recursos didácticos:

Plataforma Moodle, formularios de Google, empleo de hojas de cálculo y registro con Google Sheets, video tutoriales de Youtube, consulta de bases de datos bibliográficas y electrónicas y herramientas para la comunicación como Zoom y Teams

Formas de interacción en los ambientes de aprendizaje y de acompañamiento del trabajo independiente del estudiante:

Los conocimientos enseñados en el curso se ponen en práctica a través de la implementación de un producto de Software. Este producto se está trabajando en el marco de la Fábrica Escuela en la cual se resalta el trabajo en equipo e interdisciplinario al integrarse con estudiantes de otros cursos y roles.

Estrategias de internacionalización del currículo que se desarrollan para cumplir con las intencionalidades formativas del microcurrículo:

Se trabaja la consulta de material en inglés e igualmente se les pide el diseño de software con diagramas en inglés. Así mismo, en ocasiones, se proyectan charlas internacionales y se ejemplifica con base en el trabajo actual de empresas multinacionales.

Estrategias para abordar o visibilizar la diversidad desde la perspectiva de género, el enfoque diferencial o el enfoque intercultural:

En el curso se acuerda explícitamente el rechazo a todo tipo de Violencia Basada en Género y Violencia Sexual, así como tratos irrespetuosos o que denoten algún tipo de comportamiento excluyente o racista. Se tienen en cuenta las barreras que interfieren en el proceso de aprendizaje y se hacen los ajustes razonables. Adicionalmente, se otorgan espacios abiertos de discusión para que los estudiantes puedan socializar sus preocupaciones alrededor de estas temáticas, así como temas o problemas que puedan interferir en su buen desempeño, y se socializan situaciones que cada estudiante puede percibir diferente en su propia región.

7. EVALUACIÓN (SUGERIDA)

Concepción de evaluación, modalidades y estrategias a través de las cuales se va a orientar:

La evaluación se vincula directamente al desarrollo y alcance del proyecto en la fábrica escuela identificando dos entregables principales: un documento con el diseño en el que se aplican los conocimientos que se van trabajando en el curso, y una implementación del FrontEnd del proyecto de acuerdo a lo definido en la planificación.

Se hace uso de estrategias de Hetero-evaluación y evaluación entre pares, primando la primera a través de rúbricas de evaluación previamente socializadas.

Procesos y resultados de aprendizaje del Programa Académico que se abordan en el

curso (según el Acuerdo Académico 583 de 2021 y la Política Institucional):

RA1. Habilidad para identificar, formular y resolver problemas complejos de ingeniería aplicando los principios de la ingeniería, las ciencias y las matemáticas.

RA2. Habilidad para aplicar el diseño en ingeniería para producir soluciones que satisfagan necesidades específicas teniendo en cuenta la salud pública, la seguridad y el bienestar, así como factores globales, culturales, sociales, medioambientales y económicos.

RA3. Habilidad para comunicarse eficazmente con distintos públicos.

RA4. Habilidad para reconocer las responsabilidades éticas y profesionales en situaciones de ingeniería y de emitir juicios fundados, que deben tener en cuenta el impacto de las soluciones de ingeniería en contextos globales, económicos, medioambientales y sociales. (En este curso se realiza una medición de tipo formativo para evaluar este resultado de aprendizaje).

RA5. Habilidad para trabajar eficazmente en un equipo cuyos miembros juntos ejercen el liderazgo, crean un entorno colaborativo e inclusivo, establecen metas, planifican tareas y cumplen objetivos.

RA7. Habilidad para adquirir y aplicar nuevos conocimientos en función de las necesidades, utilizando estrategias de aprendizaje adecuadas.

Momentos de evaluación del curso y sus respectivos porcentajes	
Momento de evaluación	Porcentaje
Momento 1: Proyecto: Desarrollo de software. Entregable: RTF V1 con el diseño estructural y lógico del sistema, aplicando principios de desarrollo, patrones de diseño y principios de calidad.	15 %
Momento 2: Proyecto: Desarrollo de software. Entregable: RTF V2 con el diseño estructural y lógico del sistema, aplicando principios de desarrollo, patrones de diseño y principios de calidad.	15 %
Momento 3: Proyecto: Desarrollo de software. Entregable: RTF V3 con el diseño estructural y lógico del sistema, aplicando principios de desarrollo, patrones de diseño y principios de calidad.	20 %
Entrega 1 proyecto Fábrica Escuela	15 %
Entrega 2 proyecto Fábrica Escuela	15 %
Entrega 3 proyecto Fábrica Escuela	20 %

8. BIBLIOGRAFÍA Y OTRAS FUENTES

Cultura o zona geográfica	Bibliografía	Palabras clave
Argentina	Martin Alaimo, Martin Salias. Proyectos Ágiles con Scrum: Flexibilidad, aprendizaje, innovación y colaboración en contextos complejos. 2da Ed. Ciudad autónoma de Buenos Aires. Kleer, 2015. https:// www.amazon.com/-/ es/ Martin- Alaimo- ebook/dp/B00G5YXWQE	Marcos de trabajo ágiles, SCRUM
Estados Unidos	Applying UML and Patterns: An Introduction to Object- Oriented Analysis and Design and Iterative Development (3rd Edition). Craig Larman. Prentice Hall, 2004, 736p.	Programación, POO, UML, Patrones
Estados Unidos	El Lenguaje Unificado de Modelado. Grady Booch, James Rumbaugh, Ivar Jacobson. Addison Wesley, 1999, 432p.	UML
Estados Unidos	Guía de Arquitectura N- Capas orientada al Dominio con .NET Microsoft Application Architecture Guide, 2nd Edition https:// msdn.microsoft.com/ en- us/ library/ ee658086.aspx	Arquitectura, Diseño N- Capas, UML
Inglaterra	Software Modeling & design. UML, use cases, patterns, & software architectures. Hassan Gomma. Cambridge. 2011.	Modelado, diseño de software, UML, Casos de Uso, Patrones
España	Diseño ágil con TDD. Capítulo: 7.2. Principios S.O.L.I.D. http:// librosweb.es/ libro/ tdd/ capitulo_7/principios_solid.html Copyright (c) 2010-2013 Carlos Ble	Diseño Ágil, TDD (Test- Driven Development), Principios SOLID
Canadá	Introducing SOLID Object- Oriented Design Principles and Microsoft Unity. Regina .NET User Group. Tuesday, May 5, 2009. Protegra Inc	POO, Principios SOLID
España	Applying UML and Patterns: An Introduction to Object- Oriented Analysis and Design and Iterative Development (3rd Edition). Craig Larman. Prentice Hall, 2004, 736p	UML, Patrones
Estados Unidos	AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis 1st Edición. Raphael C. Malveau, William J. Brown, Hays W. "Skip" McCormick, Thomas J. Mowbray.	Antipatrones, arquitectura.

9. COMUNIDAD ACADÉMICA QUE PARTICIPÓ EN LA ELABORACIÓN DEL MICROCURRÍCULO

Nombres y Apellidos	Unidad académica	Formación académica	% de participación
LUZ VIVIANA COBALEDA ESTEPA	Ingeniería de Sistemas	MSc. Ingeniería de	33

		Sistemas	
ASTRID DUQUE RAMOS	Ingeniería de Sistemas	MSc. Ingeniería de Sistemas	33
Wilmer Alberto Gil Moreno	Ingeniería de Sistemas	MSc. Ingeniería de Sistemas	34

Aprobado por Comité de Carrera con acta 771 del 11 de Septiembre de 2025

Aprobado en acta de Consejo de Facultad 2507 del 24 de Septiembre de 2025